

# תרגילים – Logging

## רמה קלה – הבנה בסיסית

### תרגיל 1 – נכון / לא נכון

סמן V אם הטענה נכונה, X אם לא:

1. x print ו-logging הם שקולים לחלוטין
2. v DEBUG מתאים למידע מפורט בזמן פיתוח
3. x מותר לכתוב סיסמה בלוג אם היא מוצפנת
4. x WARNING אומר שהמערכת קרסה
5. v FileHandler שומר לוגים לקובץ
6. v logger.exception כולל גם stack trace
7. x כדאי להשתמש ברמת לוג אחת בלבד לפשטות

### תרגיל 2 – התאמת רמת לוג

לכל תיאור, ציין את רמת הלוג המתאימה (DEBUG / INFO / WARNING / ERROR):

1. משתמש התחבר בהצלחה: info
2. קובץ קונפיגורציה לא נמצא: ERROR
3. כניסה לפונקציה עם ערכי הפרמטרים: debug
4. מסד הנתונים לא זמין: error
5. מלאי מוצר נמוך מ-5 יחידות: warning
6. תהליך הזמנה הסתיים בהצלחה: info

### תרגיל 3 – זיהוי בעיות

מצא את הבעיה בכל לוג ותקן:

# לוג א

```
logger.error('User logged in successfully')
```

```
logger.info('User%s logged in successfully', user)
```

בעיה ותיקון:

# לוג ב

```
logger.info('Login', email, password)
```

```
logger.info('Login%s password$s', email, bool(password))
```

בעיה ותיקון:

# לוג ג

```
print('ERROR: payment failed')
```

```
logger.error('payment failed')
```

בעיה ותיקון:

## תרגיל 4 – מה מייצג כל שדה?

הסבר מה מייצג כל שדה בפורמט הבא:

```
'%(asctime)s | %(levelname)s | %(name)s | %(message)s'
```

%(asctime)s: חותמת זמן

%(levelname)s: logging רמת

%(name)s: שם המודול

%(message)s: הודעת השגיאה

## תרגיל 5 – השלם קוד

השלם כך שהקוד יריץ basicConfig עם INFO ויכתוב לוג אחד:

```
import logging
```

```
logging.basicConfig(
    level=logging.INFO, format='%(levelname)s | %(message)s'
```

```
    __name__)
logger = logging.getLogger()
```

```
logger.info('Application started')
```

## רמה בינונית – יישום

### תרגיל 6 – הפוך print ללוגים

שכתב את הקוד הבא כך שישתמש ב-logging במקום print:

```
def process_payment(user_id, amount):
    print(f'Starting payment for user {user_id}')
    if amount <= 0:
        print('ERROR: Invalid amount')
        return
    if amount > 10000:
        print('WARNING: Large transaction')
    print(f'Payment of {amount} completed for user {user_id}')
```

השתמש ב-INFO, WARNING, ERROR במקומות הנכונים

### תרגיל 7 – הוסף FileHandler

כתוב קוד שמגדיר logger ששומר לוגים לקובץ app.log:

1. צור logger בשם 'payments'
2. הגדר FileHandler עם utf-8 encoding
3. הוסף Formatter עם זמן, רמה ושם
4. כתוב 3 לוגים בפונקציה פשוטה

### תרגיל 8 – לוגים סביב try/except

הוסף לוגים מתאימים לקוד הבא:

```
def read_config(filepath):
    # TODO: לפני הניסיון filepath עם DEBUG הוסף
    try:
        with open(filepath) as f:
            data = f.read()
        # TODO: על הצלחה INFO הוסף
        return data
    except FileNotFoundError:
        # TODO: לוג exception הוסף
        return None
```

השתמש ב-logger.debug, logger.info, logger.exception

### תרגיל 9 – בנה פורמט JSON

כתוב פונקציה write\_structured\_log שמקבלת level, module, message ו-extra\*\* ומדפיסה JSON:

```
# פלט רצוי:
# {"timestamp": "2026-04-20T10:30:00", "level": "INFO",
#   "module": "auth", "message": "User logged in", "user_id": 42}
```

השתמש ב-datetime.utcnow().isoformat() ו- json.dumps()

## תרגיל 10 – מצא מה חסר

לכל לוג, ציין מה חסר בו ואיך לשפר:

```
logger.info('done')
```

חסר הודעה ברורה עם סוג האינפורמציה

מה חסר ואיך לשפר:

```
logger.error('failed')
```

חסר הודעה ברורה עם סוג השגיאה

מה חסר ואיך לשפר:

```
logger.info('user=%s', user_id)
```

מה טוב ומה עוד אפשר להוסיף:

מעולה שיש ארגומנטים אבל עדיין חסר פירוט מדויק מה הוא רצה לעשות

## תרגיל 11 – חלוקת אירועים לרמות

סווג כל אירוע מהמערכת לרמת הלוג הנכונה:

- אדמין נכנס למערכת הניהול info
- שירות חיצוני לא מגיב error
- פונקציית חישוב מס החלה לרוץ debug
- תעודת SSL עומדת לפוג בעוד 7 ימים warning
- הזמנה בוטלה על ידי לקוח warning
- תשלום נכשל 3 פעמים ברצף error

## רמה מתקדמת

### תרגיל 12 – שפר לוגים קיימים

הקוד הבא עובד אך הלוגים גרועים. שפר כל לוג:

```
import logging
logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)

def register_user(email, password, age):
    logger.debug('register')
    if age < 18:
        logger.error('bad')
        return
    logger.info('ok email=%s password=%s', email, password)
    logger.info('done')
```

כתוב את הגרסה המשופרת:

### תרגיל 13 – בנה מערכת לוגים מלאה

כתוב מודול logging\_setup.py שמספק:

1. פונקציה get\_logger(name) שמחזירה logger מוגדר
2. כל logger כולל StreamHandler (מסך) ו-FileHandler (קובץ)
3. Formatter אחיד עם זמן, שם, רמה והודעה
4. הדגם שימוש ב-2 מודולים שונים שמשתמשים ב-get\_logger

### תרגיל 14 – request\_id tracking

כתוב מערכת שמוסיפה request\_id לכל לוג בתוך תהליך:

1. פונקציה process\_request(request\_id, user\_id, action)
2. כל לוג בתוך הפונקציה כולל request\_id ו-user\_id
3. שים לב ל-3 לוגים לפחות: התחלה, אמצע, סוף
4. הדגם שתי קריאות לפונקציה עם request\_id שונה

השתמש ב-request\_id {extra={'request\_id': request\_id}} בכל logger call